



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Building Multi Agent Applications with Gemini ADK

Course Code: C829

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	4 October 2026	First release of the non-WSQ Learner Guide for Building Multi Agent Applications with Gemini ADK (C829), covering all 18 hands-on labs across the four course topics.	Dr Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Environment Setup.....	4
Topic 01 — Overview of Agentic AI in Gemini ADK (25%).....	6
Lab 1 — Set Up the Gemini ADK Environment and Get an API Key.....	6
Lab 2 — Build Your First ADK Agent — A Retail Banking Assistant.....	8
Lab 3 — Give an Agent Tools — Live Weather and Web Search.....	9
Lab 4 — Swap the Model — Running an ADK Agent on a Non-Gemini LLM.....	10
Topic 02 — Build A Multi Agent App with Gemini ADK (35%).....	11
Lab 5 — Give an Agent Memory — Sessions, State and the Runner.....	12
Lab 6 — Inspect the Agent Loop — Events, Tool Calls and Final Responses.....	13
Lab 7 — Multi-Agent Handoff — A Blog Content Production Team.....	14
Lab 8 — Coordinator / Dispatcher — A Customer Support Desk.....	15
Lab 9 — Workflow Agents — Sequential, Parallel, Loop and Agent-as-a-Tool.....	16
Lab 10 — Add a Guardrail — Blocking Unsafe Requests with a Callback.....	17
Lab 11 — Structured Output — Forcing Valid JSON with Pydantic.....	18
Lab 12 — Connect External Tools with MCP — StreamableHTTP and SSE.....	19
Topic 03 — Build Agentic AI RAG in Gemini ADK (25%).....	20
Lab 13 — Load, Split and Embed Documents into a Vector Store.....	21
Lab 14 — Build the Agentic RAG Agent — Retrieval as a Tool.....	22
Lab 15 — Evaluate RAG Performance — Retrieval Quality and Groundedness.....	23
Topic 04 — Build an Agentic AI App with Gemini Agent ADK and Streamlit (15%).....	24
Lab 16 — Declarative Agents — Configuring a Multi-Agent System in YAML.....	24
Lab 17 — Ship the Agent as a Web App with Streamlit.....	25
Lab 18 — Capstone — Build Your Own Multi-Agent Application.....	26
Reference — Core ADK Patterns.....	27
Reference — Evaluating a RAG Pipeline.....	30
Assessing Feasibility of an Agent Application.....	31
Continuing Your Learning.....	31
Glossary.....	31

Introduction

This Learner Guide accompanies the course Building Multi Agent Applications with Gemini ADK (C829), conducted by Tertiary Infotech Academy Pte Ltd. It provides detailed, step-by-step instructions for all 18 hands-on labs, organised by the four course topics. Every lab maps to one or more of the four course learning outcomes and is completed in Python using Google's open-source Agent Development Kit (ADK) and the Gemini model family.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. The slides give you the concepts and the shape of each lab; this guide gives you every command and every line you need to type. Work through the labs in order — each one builds on the agent patterns established by the one before it.

Keep your API keys in a .env file and never commit them to a public repository. All labs in this course use free API tiers; monitor your usage in Google AI Studio if you extend the labs beyond the classroom exercises.

Course Learning Outcomes

- LO1: Analyze the range of LLM applications using Generative AI (GAI) and identify their industrial use cases.
- LO2: Establish Google Gemini GAI designs and assess improvements on engineering processes.
- LO3: Develop LLM applications and assess its feasibility.
- LO4: Evaluate the performance effectiveness of Retrieval Augmented Generation (RAG).

Before You Start — Environment Setup

What you need

- A computer running Windows, macOS or Linux with Python 3.13 or later installed.
- The uv package manager (<https://docs.astral.sh/uv/>) — it creates the virtual environment and installs every dependency with a single command.
- A Google account, used to obtain a free Gemini API key from Google AI Studio (<https://aistudio.google.com>).
- A code editor — VS Code is recommended, but any editor will do.
- Optional free API keys used by the tool-calling labs: OpenWeather (<https://openweathermap.org>) and Tavily (<https://tavily.com>).
- An internet connection — the agents call the Gemini API over the network.

Get your Gemini API key

Open Google AI Studio at <https://aistudio.google.com> and sign in with your Google account. Click 'Get API key', then 'Create API key', and copy the key that is generated. The free tier is sufficient for every lab in this course. Treat this key like a password — anyone who has it can spend against your quota.

Install the lab environment

Clone the course repository, move into the labs folder and let uv install everything. This creates a .venv folder containing google-adk and all supporting packages.

```
git clone https://github.com/tertiarycourses/C829-Building-Multi-Agent-Applications-with-Gemini-ADK.git
```

```
cd C829-Building-Multi-Agent-Applications-with-Gemini-ADK/labs
```

```
uv sync
```

```
uv run adk --help
```

Create your .env file

Every agent loads its credentials from a .env file in the labs folder. Create it now and paste in the keys you obtained above. GOOGLE_GENAI_USE_VERTEXAI=0 tells the ADK to use the Google AI Studio API key rather than Vertex AI service-account credentials.

```
GOOGLE_GENAI_USE_VERTEXAI=0
```

```
GOOGLE_API_KEY=your-google-api-key
```

```
OPENWEATHER_API_KEY=your-openweather-key
```

```
TAVILY_API_KEY=your-tavily-key
```

The two ways to run an agent

Every lab can be run from the terminal or in the browser. Use adk run for a quick conversational check, and adk web when you want to inspect the reasoning loop, the tool calls and the handoffs between agents. adk web serves the developer UI at <http://localhost:8000>.

```
uv run adk run <agent_folder>      # terminal chat with one agent
```

```
uv run adk web                      # browser IDE: pick any agent, inspect every event
```

Conventions used in every lab

- All commands are run from the labs/ folder unless a step says otherwise.
- Commands are prefixed with uv run so they execute inside the project virtual environment.

- Placeholders such as your-google-api-key are replaced with your own values.
- Each lab folder contains an agent.py file — this is the file you read and edit.
- Press Ctrl+C to stop a running agent or the adk web server.
- If an agent cannot authenticate, check that .env is in the labs folder and that the key has no surrounding quotes or trailing spaces.

Topic 01 — Overview of Agentic AI in Gemini ADK (25%)

LLM & agentic AI foundations · Gemini model family · ADK architecture · first agent · tools

Key concepts

- A Large Language Model (LLM) is trained on very large text corpora and generates language autoregressively, one token at a time.
- Agentic AI adds reasoning, memory, tool use and autonomy on top of an LLM, so the model can act, not just answer.
- Gemini is Google DeepMind's natively multimodal model family — Pro, Flash and Nano — reasoning across text, image, video, audio and code.
- This course runs on gemini-3.8-flash, the Flash tier engineered for autonomous agents and long-horizon, multi-step reasoning — the workload a multi-agent system creates.
- Model IDs retire. Pin a specific version for reproducible training runs, or use a floating alias such as gemini-flash-latest in code you do not want to revisit; either way the model ID belongs in one MODEL constant, never scattered through the file.
- Authentication is either an AI Studio API key in GOOGLE_API_KEY, or Vertex AI via GOOGLE_CLOUD_PROJECT and GOOGLE_CLOUD_LOCATION — the agent code is identical in both cases.
- The Agent Development Kit (ADK) is Google's open-source Python framework for building, evaluating and deploying agents.
- An ADK Agent is defined by four things: a model, a name, a description and an instruction.
- Tools are ordinary Python functions the agent may call; the docstring and type hints tell the model when and how to call them.
- adk web launches a local browser IDE for chatting with an agent and inspecting every event, tool call and trace.
- LiteLlm lets an ADK agent run on non-Gemini models (OpenAI, Anthropic, Ollama) without changing agent code.

Lab 1 — Set Up the Gemini ADK Environment and Get an API Key

Learning outcome: LO1 / LO2 — establish a working Gemini ADK development environment..

Goal: Install the Agent Development Kit toolchain with uv, obtain a free Google AI Studio API key, store it safely in a .env file, and verify the Gemini model the whole course runs on. Every later lab depends on this setup working.

What you'll build

A working Python 3.13 project with google-adk installed, a validated GOOGLE_API_KEY and a confirmed gemini-3.8-flash connection. (Tools: Google AI Studio, uv, Python 3.13, google-adk, python-dotenv, gemini-3.8-flash.)

Step-by-step

1. Clone the course lab repository from GitHub

```
git clone https://github.com/tertiarycourses/C829-Building-Multi-Agent-Applications-with-Gemini-ADK.git
```

2. Change into the labs folder

```
cd C829-Building-Multi-Agent-Applications-with-Gemini-ADK/labs
```

3. Install all dependencies with uv (creates the .venv automatically)

```
uv sync
```

4. Open Google AI Studio and sign in with your Google account, then click Get API key → Create API key

5. Create a .env file in the labs folder and paste your key

```
cat > .env <<'EOF'

GOOGLE_GENAI_USE_VERTEXAI=0

GOOGLE_API_KEY=your-google-api-key

OPENWEATHER_API_KEY=your-openweather-key

TAVILY_API_KEY=your-tavily-key

EOF
```

6. Confirm the ADK command-line tool is on the path

```
uv run adk --help
```

7. Run the setup verifier — it checks Python, google-adk, the key and a live model call

```
uv run python lab01/verify_setup.py
```

8. Note the MODEL constant every lab uses, and why the ID is declared once per file

gemini-3.8-flash

9. Confirm .env is git-ignored so your API key is never committed

```
git check-ignore -v .env
```

Test it

uv run python lab01/verify_setup.py reports PASS on every check, including a live gemini-3.8-flash response, and git check-ignore confirms .env is excluded from commits.

Note: This lab is self-contained in labs/lab01/ — read verify_setup.py alongside these steps. The lab sheet for this lab is at labs/lab01/README.md. Never commit your .env file or API keys to a public repository.

Lab 2 — Build Your First ADK Agent — A Retail Banking Assistant

Learning outcome: LO1 / LO3 — define an agent from a model, name, description and instruction..

Goal: Create a single-agent banking customer-service assistant. You learn the four fields that define every ADK agent and see how the instruction alone shapes tone, scope and refusals.

What you'll build

lab02 — a Gemini-powered banking assistant that answers general banking questions and refuses to handle PINs, OTPs or full account numbers. (Tools: google-adk, Gemini 3.8 Flash, adk run, adk web.)

Step-by-step

1. Inspect the agent definition and note the four required fields

```
cat lab02/agent.py
```

2. Identify model, name, description and instruction in the Agent(...) call
3. Run the agent in the terminal

```
uv run adk run lab02
```

4. Ask an in-scope question

```
How do I reset my internet banking password?
```

5. Ask an out-of-scope question and observe the guarded refusal

```
What is my account PIN?
```

6. Launch the browser IDE and re-run the same prompts

```
uv run adk web
```

7. Open <http://localhost:8000>, select lab02, and inspect the Events tab
8. Edit the instruction to make the assistant reply only in formal English, then re-run

Test it

The agent answers general banking questions helpfully but declines to disclose or request a PIN, OTP or full account number, and the Events tab shows one LLM call per turn.

Note: This lab is self-contained in `labs/lab02/` — read `agent.py` alongside these steps. The lab sheet for this lab is at `labs/lab02/README.md`. Never commit your `.env` file or API keys to a public repository.

Lab 3 — Give an Agent Tools — Live Weather and Web Search

Learning outcome: LO1 / LO3 — extend an agent with custom function tools and evaluate tool selection..

Goal: Add two Python function tools to an agent: a live OpenWeather lookup and a Tavily web search. You see how the docstring and type hints become the tool contract the model reads.

What you'll build

lab03 — an agent that decides for itself whether a question needs the weather tool, the search tool, both, or neither. (Tools: google-adk, OpenWeather API, Tavily API, Gemini 3.8 Flash.)

Step-by-step

1. Register for a free OpenWeather API key at openweathermap.org and a Tavily key at tavily.com
2. Add both keys to `labs/.env`

```
OPENWEATHER_API_KEY / TAVILY_API_KEY
```

3. Read the two tool functions and note the docstring, the typed arguments and the dict return

```
cat lab03/agent.py
```

4. Observe that tools are attached with `tools=[get_weather, tavily_search]`
5. Run the agent

```
uv run adk run lab03
```

6. Trigger the weather tool

What is the weather in Singapore right now?

7. Trigger the search tool

What are the latest announcements about Google Gemini?

8. Ask a question needing no tool and confirm none is called

What is 15 multiplied by 12?

9. Open adk web and read the function_call and function_response events for each turn

```
uv run adk web
```

Test it

The weather question produces a `get_weather` function_call with a live temperature; the news question produces a `tavily_search` call; the arithmetic question produces no tool call at all.

Note: This lab is self-contained in `labs/lab03/` — read `agent.py` alongside these steps. The lab sheet for this lab is at `labs/lab03/README.md`. Never commit your `.env` file or API keys to a public repository.

Lab 4 — Swap the Model — Running an ADK Agent on a Non-Gemini LLM

Learning outcome: LO2 / LO4 — compare models and assess the trade-offs for an engineering process..

Goal: Use the `LiteLlm` wrapper to point the same agent at an OpenAI model, then compare it with Gemini on the same prompts to judge quality, latency and cost.

What you'll build

lab04 — one agent definition that runs on either Gemini or an OpenAI model by changing a single line. (Tools: `google-adk`, `LiteLlm`, Gemini 3.8 Flash, OpenAI GPT-4.1-mini.)

Step-by-step

1. Inspect how `LiteLlm` wraps a non-Google model

```
cat lab04/agent.py
```

2. Add your OpenAI key to `labs/.env`

```
echo 'OPENAI_API_KEY=your-openai-key' >> .env
```

3. Run the agent on the OpenAI model

```
uv run adk run lab04
```

4. Ask a reasoning question and note the answer quality and response time

Explain in three sentences why an agent needs tools.

5. Edit lab04/agent.py and replace the model with a Gemini model string

```
model='gemini-3.8-flash'
```

6. Re-run the identical prompt on Gemini

```
uv run adk run lab04
```

7. Record quality, latency and cost for both in a comparison table

Test it

The same agent runs unchanged on both providers, and you can state which model you would choose for this workload and justify it on quality, latency and cost.

Note: This lab is self-contained in labs/lab04/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab04/README.md. Never commit your .env file or API keys to a public repository.

Topic 02 — Build A Multi Agent App with Gemini ADK (35%)

Sessions & state · handoff · coordinator · Sequential · Parallel · Loop · Agent-as-a-Tool · guardrails · MCP

Key concepts

- LLMs are stateless — a Session plus a SessionService is what gives an ADK agent memory across turns.
- The Runner is the execution engine: it takes a user message, drives the agent loop and streams back Events.
- Multi-agent handoff — a root agent with sub_agents TRANSFERS control to whichever specialist matches the request, and that specialist then answers the user directly.
- The coordinator/dispatcher pattern puts a triage agent above specialised sub-agents; ADK builds the delegation tool from each sub-agent's description, so routing quality depends on how sharply those descriptions are written.
- SequentialAgent guarantees stage order in code — use it whenever stage N genuinely needs stage N-1's output, such as costing a travel itinerary that must exist first.

- `output_key` writes an agent's result into session state, and the next agent reads it back through a {placeholder} in its instruction — this is how workflow agents pass data along.
- `ParallelAgent` fans independent branches out concurrently, cutting wall-clock latency to the slowest branch rather than the sum; a fan-in agent then synthesises the branches and reconciles any conflicts.
- `LoopAgent` iterates a cycle such as writer→critic, and needs TWO stops: `max_iterations` as a hard ceiling, and an escape hatch where a tool sets `tool_context.actions.escalate = True` to exit early once the work is good enough.
- `Agent-as-a-Tool` (`AgentTool`) CALLS a specialist that returns its result to the parent, which keeps control — the opposite of `sub_agents` handoff, and the right choice when the parent still needs to compose the final answer.
- Callbacks such as `before_model_callback` implement guardrails — inspect, block or rewrite a request before it reaches the model.
- `output_schema` with a Pydantic model forces the agent to emit validated, machine-readable JSON instead of free text.
- The Model Context Protocol (MCP) is an open standard that lets an agent consume tools hosted by an external server over `StreamableHTTP` or `SSE`.

Lab 5 — Give an Agent Memory — Sessions, State and the Runner

Learning outcome: LO3 — implement session management so an agent remembers earlier turns..

Goal: Drive an agent programmatically with a Runner and an `InMemorySessionService`, so the conversation persists across turns instead of restarting on every message.

What you'll build

lab05 — a multi-turn agent that answers follow-up questions using earlier context. (Tools: `google-adk`, `Runner`, `InMemorySessionService`, `google.genai` types.)

Step-by-step

1. Read how session service, session and Runner are wired

```
cat lab05/agent.py
```

2. Note the `APP_NAME`, `USER_ID` and `SESSION_ID` that identify one conversation
3. Run the session script

```
uv run python lab05/agent.py
```

4. Ask an initial question that establishes context

```
What is the weather in Tokyo?
```

5. Ask a follow-up that only works if the agent remembers

And what about Osaka?

6. Comment out the session creation and re-run to observe the failure
7. Restore the session code and confirm continuity is back

Test it

The follow-up 'And what about Osaka?' is understood as a weather question without repeating the word weather — and stops working when the session is removed.

Note: This lab is self-contained in labs/lab05/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab05/README.md. Never commit your .env file or API keys to a public repository.

Lab 6 — Inspect the Agent Loop — Events, Tool Calls and Final Responses

Learning outcome: LO3 / LO4 — analyse the agent execution loop and evaluate its behaviour..

Goal: Stream the Event objects the Runner emits and learn to read an agent trace: which event carries the tool call, which carries the tool result, and which is the final response.

What you'll build

lab06 — an instrumented agent that prints every event in its reasoning loop. (Tools: google-adk, Runner, Events API, InMemorySessionService.)

Step-by-step

1. Read the event-handling loop and find is_final_response()

```
cat lab06/agent.py
```

2. Run the interaction script

```
uv run python lab06/agent.py
```

3. Send a prompt that requires a tool

Find the weather in Singapore and summarise recent AI news.

4. In the printed trace, identify the function_call event
5. Identify the function_response event carrying the tool's return value
6. Identify the final response event and note how many LLM calls one turn actually took

7. Explain why a two-tool question produces more events than a one-tool question

Test it

You can point to the `function_call`, the `function_response` and the final response in the trace, and state the number of LLM round-trips the turn consumed.

Note: This lab is self-contained in `labs/lab06/` — read `agent.py` alongside these steps. The lab sheet for this lab is at `labs/lab06/README.md`. Never commit your `.env` file or API keys to a public repository.

Lab 7 — Multi-Agent Handoff — A Blog Content Production Team

Learning outcome: LO3 — implement agent-to-agent delegation with `sub_agents`..

Goal: Build a realistic editorial team: a managing editor hands off to a researcher, who gathers current facts with web search and then hands off to a writer who drafts the post. Every handoff is driven by the description field — the core ADK delegation pattern.

What you'll build

lab07 — a three-stage content pipeline that turns a topic request into a publish-ready, fact-grounded blog post. (Tools: `google-adk`, `sub_agents`, `google_search`, Gemini 3.8 Flash.)

Step-by-step

1. Read the three agents and the `sub_agents` chain

```
cat lab07/agent.py
```

2. The description field is what drives the parent's handoff decision
3. Run the agent in the browser IDE

```
uv run adk web
```

4. Select lab07 and commission a post

Write a blog post about AI adoption in Singapore SMEs

5. In the Events tab, find the `transfer_to_agent` call into `researcher_agent`
6. Confirm the researcher's `google_search` calls, then the transfer into `writer_agent`
7. Check the finished post cites only facts that appear in the research brief
8. Weaken the `writer_agent` description to one vague word, re-run and observe the handoff become unreliable, then restore it

Test it

One topic request produces a research brief followed by a cited blog post, and the Events tab shows two `transfer_to_agent` calls in order.

Note: This lab is self-contained in `labs/lab07/` — read `agent.py` alongside these steps. The lab sheet for this lab is at `labs/lab07/README.md`. Never commit your `.env` file or API keys to a public repository.

Lab 8 — Coordinator / Dispatcher — A Customer Support Desk

Learning outcome: LO3 — design a coordinator agent routing to specialised sub-agents..

Goal: Build a production-shaped support desk where a triage agent dispatches each ticket to the billing, technical or returns specialist — each with its own instruction and its own tools against order and refund systems.

What you'll build

lab08 — a triage coordinator with three tool-equipped specialists that routes real support tickets. (Tools: `google-adk`, `sub_agents`, `coordinator/dispatcher` pattern, function tools.)

Step-by-step

1. Read the three specialists, their tools and the triage coordinator

```
cat lab08/agent.py
```

2. Compare the three descriptions and note how each states its routing trigger
3. Launch the web IDE and select lab08

```
uv run adk web
```

4. Send a billing ticket and confirm it routes to `billing_agent`

```
I was charged twice for order SG-10482
```

5. Send a technical ticket

```
My wireless keyboard will not pair with my laptop
```

6. Send a returns ticket and watch the tool calls

```
Where is order SG-10915 and can I still refund it?
```

7. Send a ticket that spans two queues and observe how triage resolves it

```
My keyboard is faulty and I want my money back
```

8. Add a fourth specialist of your own to sub_agents and test the routing

Test it

Each ticket reaches the matching specialist as a transfer_to_agent event, the returns ticket triggers lookup_order and check_refund_eligibility, and your fourth specialist routes.

Note: This lab is self-contained in labs/lab08/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab08/README.md. Never commit your .env file or API keys to a public repository.

Lab 9 — Workflow Agents — Sequential, Parallel, Loop and Agent-as-a-Tool

Learning outcome: LO3 / LO2 — orchestrate multi-agent workflows with all four ADK collaboration patterns..

Goal: Work through the four ADK workflow patterns on realistic systems: a SequentialAgent travel planner, a ParallelAgent market-research desk with a synthesis step, a LoopAgent writer/critic refinement cycle, and an AgentTool bilingual report writer. The lab makes the decisive distinction explicit — when order is enforced by code vs by the model, and when a specialist takes control (handoff) vs returns a result (AgentTool).

What you'll build

lab09 — four runnable multi-agent systems: agent.py (sequential travel planner), agent_parallel.py (market research), agent_loop.py (copy refinement), agent_as_tool.py (bilingual report writer). (Tools: google-adk, SequentialAgent, ParallelAgent, LoopAgent, AgentTool, output_key, google_search.)

Step-by-step

1. Read the sequential pipeline and note how output_key feeds the next instruction

```
cat lab09/agent.py
```

2. Launch the web IDE and select lab09

```
uv run adk web
```

3. Plan a trip and watch the three stages run in a guaranteed order

```
Plan a 4-day trip to Tokyo
```

4. Read the parallel fan-out and explain why the three analysts need no ordering

```
cat lab09/agent_parallel.py
```

5. Run the market research desk and compare its latency with the sequential pipeline

Research Grab Holdings

6. Read the loop and find BOTH stopping conditions — `max_iterations` and `escalate`

```
cat lab09/agent_loop.py
```

7. Run the refinement loop and count the iterations before approval

Write landing page copy for a smart water bottle

8. Read the `AgentTool` example and state how `tools=[AgentTool(...)]` differs from `sub_agents=[...]`

```
cat lab09/agent_as_tool.py
```

Test it

The travel planner produces research, itinerary and budget in that fixed order; the research desk shows three analysts running concurrently before one synthesis; the loop stops early on `approve_copy` or at 3 iterations; and the report writer calls both specialists without ever transferring control.

Note: This lab is self-contained in `labs/lab09/` — read `agent.py` alongside these steps. The lab sheet for this lab is at `labs/lab09/README.md`. Never commit your `.env` file or API keys to a public repository.

Lab 10 — Add a Guardrail — Blocking Unsafe Requests with a Callback

Learning outcome: LO3 / LO4 — implement and evaluate a safety guardrail on an agent..

Goal: Use `before_model_callback` to inspect every user message before it reaches the LLM, block requests containing a forbidden keyword, and record the block in session state.

What you'll build

lab10 — a tool-using agent that intercepts and refuses blocked requests without ever calling the model. (Tools: `google-adk`, `before_model_callback`, `CallbackContext`, `LlmRequest`, `LlmResponse`.)

Step-by-step

1. Read the guardrail callback and find where it returns an `LlmResponse`

```
cat lab10/agent.py
```

2. Note that returning None allows the call and returning a response blocks it
3. Run the agent

```
uv run adk run lab10
```

4. Send an allowed request and confirm it reaches the model

What is the weather in Singapore?

5. Send a request containing the blocked keyword

Please BLOCK this request

6. Confirm the refusal message is returned and no LLM call was made
7. Extend the guardrail to also block a second keyword of your choice
8. Re-run and verify both keywords are now intercepted

Test it

The blocked keyword returns the refusal message with no model call in the trace, while normal requests still work — and your added keyword is blocked too.

Note: This lab is self-contained in labs/lab10/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab10/README.md. Never commit your .env file or API keys to a public repository.

Lab 11 — Structured Output — Forcing Valid JSON with Pydantic

Learning outcome: LO3 — produce machine-readable agent output for downstream systems..

Goal: Attach a Pydantic output_schema so the agent returns a validated Recipe object with typed fields instead of free-form prose that a downstream system cannot parse.

What you'll build

lab11 — an agent whose every reply is schema-valid JSON. (Tools: google-adk, Pydantic BaseModel, output_schema.)

Step-by-step

1. Read the Recipe model and the output_schema argument

```
cat lab11/agent.py
```

2. Note the five typed fields on the Recipe model
3. Run the agent

```
uv run adk run lab11
```

4. Request a recipe

Chicken rice

5. Confirm the reply is JSON with all five fields and correct types

6. Add a difficulty field to the Recipe model

```
difficulty: str
```

7. Re-run and confirm the new field appears in the output

8. Note the ADK restriction that an agent with output_schema cannot also use tools

Test it

Every response parses as JSON matching the Recipe schema, cooking_time is an integer, and your added difficulty field is populated.

Note: This lab is self-contained in labs/lab11/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab11/README.md. Never commit your .env file or API keys to a public repository.

Lab 12 — Connect External Tools with MCP — StreamableHTTP and SSE

Learning outcome: LO3 / LO2 — integrate third-party tools through the Model Context Protocol..

Goal: Connect an ADK agent to a remote MCP server so it can use tools it does not own, using both the StreamableHTTP and SSE transports.

What you'll build

lab12 — agent.py (StreamableHTTP) and agent_sse.py (SSE), whose toolset is discovered at runtime from an MCP server. (Tools: google-adk, McpToolset, StreamableHTTPConnectionParams, SseConnectionParams, n8n MCP server.)

Step-by-step

1. Read how McpToolset discovers tools from the server URL

```
cat lab12/agent.py
```

2. Run the StreamableHTTP MCP agent and read the printed tool list

```
uv run python lab12/agent.py
```

3. Send a request that exercises one of the discovered tools

4. Compare the SSE variant and note that only the connection params differ

```
cat lab12/agent.py
```

5. Explain when SSE is preferred over StreamableHTTP
6. Point the MCP_SERVER_URL at a different MCP server and re-run
7. Confirm the agent's available tools change without any change to the agent code

Test it

The agent prints the tools discovered from the MCP server and successfully invokes one; changing the server URL changes the toolset with no code edit.

Note: This lab is self-contained in labs/lab12/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab12/README.md. Never commit your .env file or API keys to a public repository.

Topic 03 — Build Agentic AI RAG in Gemini ADK (25%)

RAG pipeline · loading & splitting · embeddings · vector stores · retrieval · grounded answers

Key concepts

- Retrieval Augmented Generation (RAG) grounds an LLM in your own documents, reducing hallucination and keeping answers current.
- The RAG pipeline is: load → split → embed → store → retrieve → generate.
- Loaders convert PDF, HTML, CSV, Word and web sources into a common document object.
- Splitting breaks documents into chunks small enough to embed and retrieve precisely, with overlap to preserve context.
- An embedding maps text to a dense vector so that semantically similar text sits close together in vector space.
- A vector database (Chroma, FAISS, Pinecone, Milvus) stores embeddings and answers nearest-neighbour similarity queries.
- Similarity search returns the closest chunks; Maximum Marginal Relevance (MMR) trades a little similarity for diversity.
- In ADK, retrieval is wrapped as a tool so the agent decides when to consult the knowledge base.
- RAG performance is evaluated on retrieval quality, groundedness/faithfulness, answer relevance and latency.

Lab 13 — Load, Split and Embed Documents into a Vector Store

Learning outcome: LO4 — build the ingestion half of a RAG pipeline and inspect the embeddings..

Goal: Take two product PDFs, extract their text, split it into retrievable chunks with page metadata, embed them and persist the vectors in a local Chroma database.

What you'll build

A persistent Chroma collection holding the chunked and embedded air-fryer product and warranty manuals. (Tools: ChromaDB, pypdf, google-adk, Gemini embeddings.)

Step-by-step

1. Inspect the two source PDFs the agent will be grounded in

```
ls lab13/*.pdf
```

2. Read the extraction function and note the page number kept as metadata

```
cat lab13/agent.py
```

3. Identify the chunking rule and the minimum chunk length filter

4. Run the agent once to trigger ingestion into ChromaDB

```
uv run adk run lab13
```

5. Confirm the persistent vector store was created on disk

```
ls lab13/chroma_db
```

6. Explain why chunks shorter than 50 characters are discarded

7. Change the chunk rule to split on single newlines, delete chroma_db, and re-ingest

```
rm -rf lab13/chroma_db
```

8. Compare the resulting chunk count and note the effect on retrieval precision

Test it

lab13/chroma_db exists and contains a populated air_fryer_docs collection, and you can state how the chunking rule changed the number of stored chunks.

Note: This lab is self-contained in labs/lab13/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab13/README.md. Never commit your .env file or API keys to a public repository.

Lab 14 — Build the Agentic RAG Agent — Retrieval as a Tool

Learning outcome: LO4 / LO3 — wrap retrieval as a tool so the agent grounds its answers in your documents..

Goal: Complete the RAG loop: expose the vector search as an ADK tool, let the agent decide when to retrieve, and require it to cite the source document and page.

What you'll build

lab14 — a grounded question-answering agent over the air-fryer manuals with citations. (Tools: google-adk, ChromaDB, similarity search, Gemini 3.8 Flash.)

Step-by-step

1. Read the retrieval tool and how the query is embedded

```
cat lab14/agent.py
```

2. Answers must come only from retrieved context
3. Run the RAG agent

```
uv run adk run lab14
```

4. Ask a question answerable from the product manual

```
What temperature should I use to air fry chicken wings?
```

5. Ask a question answerable only from the warranty document

```
How long is the warranty period and what does it exclude?
```

6. Ask a question the documents do not cover and confirm the agent declines rather than invents

```
What is the share price of the manufacturer?
```

7. In adk web, inspect the retrieval function_response to see the chunks that were retrieved

```
uv run adk web
```

8. Increase the number of retrieved chunks (n_results) and re-run the same questions

Test it

Both document questions are answered with the correct source and page cited, the out-of-scope question is refused rather than hallucinated, and you can see the retrieved chunks.

Note: This lab is self-contained in labs/lab14/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab14/README.md. Never commit your .env file or API keys to a public repository.

Lab 15 — Evaluate RAG Performance — Retrieval Quality and Groundedness

Learning outcome: LO4 — evaluate the performance effectiveness of a RAG implementation..

Goal: Build a small evaluation set and score your RAG agent on retrieval hit rate, groundedness, answer relevance and latency, then tune one parameter and measure the improvement.

What you'll build

A completed RAG evaluation table with a before-and-after comparison for one tuned parameter. (Tools: lab15, ChromaDB, similarity search vs MMR.)

Step-by-step

1. Write eight test questions: six answerable from the PDFs, two deliberately out of scope
2. For each question record whether the correct chunk was retrieved (retrieval hit)
3. Record whether every claim is grounded in the retrieved text
4. Record whether the answer actually addresses the question (relevance) and its response time
5. Compute the hit rate, groundedness rate and mean latency across the eight questions
6. Tune one parameter — chunk size, n_results, or switch similarity search to MMR
7. Re-run the same eight questions and recompute all three metrics
8. State which parameter change you would keep and justify it with your numbers

Test it

A completed evaluation table with before-and-after figures for retrieval hit rate, groundedness and latency, plus a written, evidence-based tuning recommendation.

Note: This lab is self-contained in labs/lab15/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab15/README.md. Never commit your .env file or API keys to a public repository.

Topic 04 — Build an Agentic AI App with Gemini Agent ADK and Streamlit (15%)

From agent to product · Streamlit chat UI · async Runner · deployment & feasibility

Key concepts

- A production agent app needs a user interface, session handling, error handling and API-key management.
- Streamlit turns a Python script into a web app, making it the fastest route from an ADK agent to a usable product.
- `st.session_state` persists the ADK SessionService and the chat history across Streamlit re-runs.
- The ADK Runner is async, so a Streamlit app drives it through `asyncio.run()` inside the request handler.
- Secrets belong in `.env` and never in source control or in the repository you push to GitHub.
- Assessing feasibility means weighing accuracy, latency, token cost, maintainability and governance against the business benefit.

Lab 16 — Declarative Agents — Configuring a Multi-Agent System in YAML

Learning outcome: LO2 / LO3 — separate agent configuration from code to improve maintainability..

Goal: Define a five-specialist transport assistant entirely in YAML config files, with no Python agent code, and assess what this buys you for maintainability and governance.

What you'll build

lab16 — a root agent with MRT, bus, taxi, bike and walk specialists, all declared in YAML. (Tools: google-adk YAML config, LlmAgent, Gemini 3.8 Flash.)

Step-by-step

1. List the YAML files and note one file per agent

```
ls lab16/
```

2. Read the root agent config and its `sub_agents config_path` list

```
cat lab16/root_agent.yaml
```

3. Read one specialist config and compare its fields to the `Python Agent(...)` arguments

```
cat lab16/mrt_agent.yaml
```

4. Run the YAML-configured agent

```
uv run adk web
```

5. Ask an MRT question and confirm it routes to mrt_agent

What is the fastest MRT route from Bugis to Woodlands?

6. Ask a cycling question and confirm it routes to bike_agent
7. Add a sixth specialist by creating a new YAML file and referencing it in root_agent.yaml
8. Re-run and confirm the new specialist is routed to without touching any Python file

Test it

All five specialists route correctly, and your sixth agent works after editing YAML only — no Python file was modified.

Note: This lab is self-contained in labs/lab16/ — read its YAML config files alongside these steps. The lab sheet for this lab is at labs/lab16/README.md. Never commit your .env file or API keys to a public repository.

Lab 17 — Ship the Agent as a Web App with Streamlit

Learning outcome: LO3 — deploy a multi-agent system behind a usable chat interface..

Goal: Wrap the sequential transport agent in a Streamlit chat UI, wiring the async ADK Runner and persisting the session across Streamlit re-runs so the conversation survives.

What you'll build

lab17 — a browser chat application backed by the multi-agent transport workflow.
(Tools: Streamlit, google-adk Runner, InMemorySessionService, asyncio.)

Step-by-step

1. Read how the ADK session service is stored in st.session_state

```
cat lab17/app.py
```

2. The async Runner is driven by asyncio.run in the handler
3. Launch the Streamlit application

```
uv run streamlit run lab17/app.py
```

4. Open the app in the browser and submit a journey

From Tampines to Orchard Road

5. Send a follow-up question and confirm the conversation history is retained

What if I cycle instead?

6. Refresh the browser and observe what happens to the session
7. Change the page title and icon in `st.set_page_config` and reload
8. Add a sidebar Clear chat button that resets `st.session_state.messages`

Test it

The Streamlit app answers a journey query, retains history across follow-up turns, and your Clear chat button empties the conversation without restarting the server.

Note: This lab is self-contained in `labs/lab17/` — read `agent.py` and `app.py` alongside these steps. The lab sheet for this lab is at `labs/lab17/README.md`. Never commit your `.env` file or API keys to a public repository.

Lab 18 — Capstone — Build Your Own Multi-Agent Application

Learning outcome: LO1 / LO2 / LO3 / LO4 — design a multi-agent solution and assess its feasibility..

Goal: In groups of three to five, design and build a multi-agent ADK application for a use case in your own industry, then present it with a feasibility assessment.

What you'll build

A working multi-agent application with at least three agents, at least two tools, session memory and a documented feasibility assessment. (Tools: `google-adk`, `sub_agents` or `SequentialAgent`, custom tools, optional RAG and Streamlit.)

Step-by-step

1. Form a group of three to five and choose an industrial use case from your own sector
2. Identify the specialist roles and draw the agent topology — coordinator or sequential
3. List the tools each agent needs and which are custom functions versus built-in
4. Build the agents, using the closest lab as your template
5. Add session memory so the application handles multi-turn conversations
6. Add a guardrail or a structured output schema appropriate to your use case
7. Test with at least five realistic prompts and record where the agent fails

8. Assess feasibility: accuracy, latency, token cost, maintainability and governance risk
9. Present the application and the feasibility assessment to the class in five minutes

Test it

A running application with three or more agents and two or more tools, demonstrated live, plus a feasibility assessment stating whether you would recommend production deployment and on what evidence.

Note: This lab is self-contained in labs/lab18/ — read agent.py alongside these steps. The lab sheet for this lab is at labs/lab18/README.md. Never commit your .env file or API keys to a public repository.

Reference — Core ADK Patterns

These are the patterns you will reuse in every agent you build after this course. Each one appears in at least one lab; this section collects them in one place for quick reference.

Defining an agent

An agent is defined by a model, a name, a description and an instruction. The description is what a parent agent reads when deciding whether to hand off to it, so write it as a routing trigger.

```
from google.adk.agents import Agent
```

```
root_agent = Agent(
    model='gemini-3.8-flash',
    name='root_agent',
    description='A helpful assistant for user questions.',
    instruction='You are a professional assistant. Answer clearly and concisely.',
)
```

Adding a tool

A tool is an ordinary Python function. The docstring and the type hints are the contract the model reads to decide when and how to call it — write them carefully. Return a dict with a status field so the model can distinguish success from failure.

```
def get_weather(city: str) -> dict:
    """Retrieves the current weather for a specified city.

    Args:
        city (str): The name of the city.

    Returns:
        dict: status and result or error msg.
    """
    return {'status': 'success', 'report': '...'}
```

```
agent = Agent(..., tools=[get_weather])
```

Multi-agent handoff

Attach specialists with sub_agents. The parent transfers control to whichever sub-agent's description best matches the request.

```
root_agent = Agent(
    name='root_agent',
    model=MODEL,
    instruction='Route each question to the right specialist.',
    sub_agents=[math_tutor_agent, physics_tutor_agent,
history_tutor_agent],
)
```

Sequential workflow

Use SequentialAgent when the stages must run in a fixed order and each stage's output feeds the next.

```
from google.adk.agents import SequentialAgent
```

```
workflow = SequentialAgent(
    name='workflow_agent',
```

```

        description='Collects input, then researches it, then reports.',
        sub_agents=[input_agent, research_agent, report_agent],
    )

```

Sessions and the Runner

An LLM is stateless. A Session plus a SessionService is what gives the agent memory across turns, and the Runner is the engine that executes one turn and streams back events.

```

from google.adk.runners import Runner

from google.adk.sessions import InMemorySessionService

session_service = InMemorySessionService()

await session_service.create_session(app_name=APP, user_id=USER,
session_id=SID)

runner = Runner(agent=root_agent, app_name=APP,
session_service=session_service)

```

Guardrail callback

`before_model_callback` runs before every model call. Return `None` to allow the call, or return an `LlmResponse` to block it and answer directly.

```

def block_keyword_guardrail(callback_context, llm_request):

    if 'BLOCK' in last_user_message.upper():

        return LlmResponse(content=types.Content(

            role='model', parts=[types.Part(text='I cannot process this
request.')])))

    return None

```

```

agent = Agent(..., before_model_callback=block_keyword_guardrail)

```

Structured output

Attach a Pydantic model as `output_schema` to force validated JSON. Note that an agent with an `output_schema` cannot also use tools.

```

from pydantic import BaseModel

```

```
class Recipe(BaseModel):
    title: str
    ingredients: list[str]
    cooking_time: int

agent = Agent(..., output_schema=Recipe)
```

Switching models with LiteLlm

LiteLlm lets the same agent code run on a non-Gemini model.

```
from google.adk.models.lite_llm import LiteLlm

agent = Agent(model=LiteLlm(model='openai/gpt-4.1-mini'), ...)
```

Reference — Evaluating a RAG Pipeline

Topic 3 asks you to evaluate the performance effectiveness of Retrieval Augmented Generation. Use these four measures, applied to a fixed set of test questions, so that any change you make can be compared like for like.

- Retrieval hit rate — for what fraction of questions did the retriever return the chunk that actually contains the answer? This is the ceiling on everything downstream: if retrieval misses, the answer cannot be right.
- Groundedness (faithfulness) — is every claim in the answer supported by the retrieved text? An answer that is correct but not supported by the retrieved chunks is still a hallucination risk.
- Answer relevance — does the answer actually address the question asked, rather than a related one?
- Latency and cost — how long does a full retrieve-and-generate turn take, and how many tokens does it consume? A pipeline that is accurate but too slow or too expensive is not feasible in production.

Tune one parameter at a time — chunk size, chunk overlap, the number of retrieved chunks (`n_results`), or similarity search versus Maximum Marginal Relevance — and re-measure. Smaller chunks usually improve retrieval precision but can cut a fact in half; larger chunks preserve context but dilute the embedding. MMR trades a little similarity for diversity, which helps when your documents contain many near-duplicate passages.

Also test what the agent does with a question the documents cannot answer. A well-built RAG agent declines; a poorly grounded one invents an answer. This behaviour is part of the evaluation.

Assessing Feasibility of an Agent Application

Learning outcome LO3 requires you to assess the feasibility of an LLM application. Use these dimensions when you present your capstone, and when you propose an agent at work.

- Accuracy and reliability — how often does the agent produce a correct, complete answer? Where does it fail, and is that failure mode acceptable in your context?
- Latency — is the response time acceptable to the end user? Multi-agent pipelines make several LLM calls per turn and are slower than a single agent.
- Token cost — estimate the cost per conversation and multiply by the expected volume. Sessions grow the context with every turn, so cost per turn rises through a long conversation.
- Maintainability — who updates the instructions and tools when the business process changes? YAML-configured agents can be edited without touching Python.
- Governance and risk — what happens if the agent is wrong, leaks data, or is prompted maliciously? Guardrails, structured output and human review reduce this exposure.
- Alternatives — would a simpler solution (a single agent, a fixed script, or no AI at all) meet the need at lower cost and risk?

Continuing Your Learning

- First pass: complete every lab, reading agent.py alongside the steps in this guide.
- Second pass: rebuild the key labs from a blank file until the agent pattern is automatic.
- Read the official ADK documentation at <https://google.github.io/adk-docs/> for the full API surface.
- Extend your capstone with RAG, a guardrail and a Streamlit interface.
- Deploy an agent to Cloud Run or Vertex AI Agent Engine to take it to production.
- Explore related courses at Tertiary Infotech to broaden your Generative AI skill set.

Glossary

- **Agent** — An LLM configured with a name, description, instruction and optionally tools and sub-agents, able to reason and act rather than only answer.
- **Agentic AI** — AI systems that add reasoning, memory, tool use and autonomy on top of a language model so they can complete tasks.

- **ADK** — Agent Development Kit — Google's open-source Python framework for building, evaluating and deploying agents.
- **LLM** — Large Language Model — a model trained on very large text corpora that generates language one token at a time.
- **Gemini** — Google DeepMind's natively multimodal model family (Ultra, Pro, Flash, Nano) used throughout this course.
- **Instruction** — The system prompt that defines an agent's role, rules, tone and boundaries.
- **Description** — A short statement of what an agent is for; a parent agent reads it to decide whether to hand off.
- **Tool** — A Python function an agent may call to fetch data or take an action; its docstring and type hints form the contract.
- **Function calling** — The mechanism by which the model requests that a named tool be run with specific arguments.
- **Session** — The stored history of one conversation, which gives an otherwise stateless model memory across turns.
- **SessionService** — The component that creates and stores sessions — InMemorySessionService in these labs.
- **Runner** — The ADK execution engine that takes a user message, drives the agent loop and streams back events.
- **Event** — One step in the agent loop — a model response, a function call, a function response or a final answer.
- **Sub-agent** — A specialist agent that a parent agent can transfer control to.
- **Handoff** — The transfer of control from one agent to another, recorded as a transfer_to_agent event.
- **SequentialAgent** — A workflow agent that runs its sub-agents in a fixed order, passing each output to the next.
- **ParallelAgent** — A workflow agent that runs its sub-agents concurrently.
- **Callback** — A hook such as before_model_callback that runs at a defined point in the agent loop.
- **Guardrail** — A policy enforced by a callback that inspects, blocks or rewrites a request before it reaches the model.
- **Structured output** — Forcing an agent to emit JSON validated against a Pydantic schema via output_schema.
- **MCP** — Model Context Protocol — an open standard letting an agent consume tools hosted by an external server.
- **LiteLlm** — An ADK wrapper that lets an agent run on non-Gemini models such as OpenAI or Anthropic.
- **RAG** — Retrieval Augmented Generation — grounding an LLM's answers in retrieved passages from your own documents.
- **Chunking** — Splitting a document into passages small enough to embed and retrieve precisely.

- **Embedding** — A dense vector representation of text, positioned so that semantically similar text is nearby.
- **Vector database** — A store for embeddings that answers nearest-neighbour similarity queries — Chroma in these labs.
- **Similarity search** — Retrieving the chunks whose embeddings are closest to the query embedding.
- **MMR** — Maximum Marginal Relevance — a retrieval strategy that trades some similarity for diversity of results.
- **Groundedness** — The degree to which every claim in an answer is supported by the retrieved source text.
- **Hallucination** — A fluent but unsupported or fabricated statement produced by a language model.
- **Temperature** — A sampling parameter controlling randomness — low is focused and repeatable, high is creative.
- **Top-K / Top-P** — Sampling parameters that restrict the candidate next tokens by rank or by cumulative probability.
- **Streamlit** — A Python framework that turns a script into a web application, used to give an agent a chat UI.